Credit-Risk-Modeling-with-Ensemble-Learning-for-Loan-Default-Detection

DETAILED REPORT

INTRODUCTION

Financial institutions issue thousands of loans daily, and it is very important to perform an accurate assessment of the credit risk. Nowadays, this has become a serious concern because those institutions are facing a major challenge in identifying which loan applicants are likely to default. Predicting credit risk early surely helps lenders reduce losses, improve approval decisions, optimize the interest rate, and further maintain regulatory compliance.

In this project, we are dealing with a real-world business concern by analyzing a credit-risk dataset, and based on that, we will build a machine learning model that will predict whether a person will default on a loan or not. This model will be designed by taking into consideration several parameters like person demographics, employment, financial information, credit history, along with loan attributes and different outcomes.

MAIN RESEARCH QUESTION:

Can we build a reliable machine learning model to classify default and non-default borrowers based on the demographic, financial and credit-related features? Also, Which borrower-related factors most strongly predict loan default ?

This project answers all these questions using exploratory data analysis, data preprocessing, feature engineering, and multiple machine learning models to identify the strongest predictors for credit default risk.

METHODOLOGY

1. **Data Collection:**
   The dataset is collected from Kaggle.com and its name is "Credit Risk Dataset". It contains 32,581 records and 12 columns in which "loan_status" is the target variable and remaining 11 columns are the independent features. Here are the features along with their description:

Features	Data Type	Description
person_age	Numeric(int)	Age of the individual
person_income	Numeric(int)	Annual income of the individual
person_home_ownership	Categorical	Type of home ownership(rent,mortgage,own,other)
person_emp_length	Numeric(int)	Employment length (in years)
loan_intent	Categorical	Intent behind the loan application
loan_grade	Categorical	Grade assigned based on borrower creditworthiness
loan_amnt	Numeric(int)	Loan amount requested
loan_int_rate	Numeric(float)	Interest rate associated with loan
loan_status	Binary(0/1)	Target variable: 0 [non-default] or 1[default]
loan_percent_income	Numeric(float)	Percentage of income
cb_person_defaullt_on_file	Binary(char)	Historical default
cb_person_cred_hist_length	Numeric(int)	Length of credit history

We imported the dataset csv file using pandas DataFrame, inspected the structure, and stored it for further processing. Further, we verified the total rows, columns, and their datatypes, along with the examination of basic statistics for the numerical columns. Here is the visual representation of all the performed steps:

Final_Project_Credit_Risk_Prediction.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Importing Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
credit_risk_df = pd.read_csv("/content/credit_risk_dataset.csv")
```

Exploratory Data Analysis(EDA)

```
credit_risk_df.head()
```

	person_age	person_income	person_home_ownership	person_emp_length	loan_intent	loan_grade	loan_amnt	loan_int_rate	loan_status	loan_percent_income	cb_person_default_on_file	cb_person_cred_hist_length
0	22	59000	RENT	123.0	PERSONAL	D	35000	16.02	1	0.59	Y	3
1	21	9600	OWN	5.0	EDUCATION	B	1000	11.14	0	0.10	N	2
2	25	9600	MORTGAGE	1.0	MEDICAL	C	5500	12.87	1	0.57	N	3
3	23	65500	RENT	4.0	MEDICAL	C	35000	15.23	1	0.53	N	2
4	24	54400	RENT	8.0	MEDICAL	C	35000	14.27	1	0.55	Y	4

Next steps: Generate code with credit_risk_df New interactive sheet

```
credit_risk_df.tail()
```

	person_age	person_income	person_home_ownership	person_emp_length	loan_intent	loan_grade	loan_amnt	loan_int_rate	loan_status	loan_percent_income	cb_person_default_on_file	cb_person_cred_hist_length
32576	57	53000	MORTGAGE	1.0	PERSONAL	C	5800	13.16	0	0.11	N	30
32577	54	120000	MORTGAGE	4.0	PERSONAL	A	17625	7.49	0	0.15	N	19
32578	65	76000	RENT	3.0	HOMEIMPROVEMENT	B	35000	10.99	1	0.46	N	28
32579	56	150000	MORTGAGE	5.0	PERSONAL	B	15000	11.48	0	0.10	N	26
32580	66	42000	RENT	2.0	MEDICAL	B	6475	9.99	0	0.15	N	30

Figure 1: Samples of Dataset

```
credit_risk_df.shape
```

```
(32581, 12)
```

```
credit_risk_df.describe()
```

	person_age	person_income	person_emp_length	loan_amnt	loan_int_rate	loan_status	loan_percent_income	cb_person_cred_hist_length
count	32581.000000	3.258100e+04	31686.000000	32581.000000	29465.000000	32581.000000	32581.000000	32581.000000
mean	27.734600	6.607485e+04	4.789686	9589.371106	11.011695	0.218164	0.170203	5.804211
std	6.348078	6.198312e+04	4.142630	6322.086646	3.240459	0.413006	0.106782	4.055001
min	20.000000	4.000000e+03	0.000000	500.000000	5.420000	0.000000	0.000000	2.000000
25%	23.000000	3.850000e+04	2.000000	5000.000000	7.900000	0.000000	0.090000	3.000000
50%	26.000000	5.500000e+04	4.000000	8000.000000	10.990000	0.000000	0.150000	4.000000
75%	30.000000	7.920000e+04	7.000000	12200.000000	13.470000	0.000000	0.230000	8.000000
max	144.000000	6.000000e+06	123.000000	35000.000000	23.220000	1.000000	0.830000	30.000000

```
credit_risk_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32581 entries, 0 to 32580
Data columns (total 12 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   person_age                  32581 non-null  int64
 1   person_income               32581 non-null  int64
 2   person_home_ownership       32581 non-null  object
 3   person_emp_length           31686 non-null  float64
 4   loan_intent                 32581 non-null  object
 5   loan_grade                  32581 non-null  object
 6   loan_amnt                   32581 non-null  int64
 7   loan_int_rate               29465 non-null  float64
 8   loan_status                 32581 non-null  int64
 9   loan_percent_income         32581 non-null  float64
 10  cb_person_default_on_file   32581 non-null  object
 11  cb_person_cred_hist_length  32581 non-null  int64
dtypes: float64(3), int64(5), object(4)
memory usage: 3.0+ MB
```

Figure 2: Dataset Info & Basic Statistics

2. Exploratory Data Analysis:

It started with exploring both the numerical and the categorical features, in order to understand the various trends in income, loan amount, employment length, home ownership, and loan purpose. Firstly, we made the confusion matrix for all the numerical features to check if there is any strong correlation between any two features or between the one feature and the target variable. We didn't find any direct strong relation from that plot. After that, we performed univariant analysis for categorical features to check any pattern or trend for all the unique values of each feature. Here is the visual representation of all the plots:

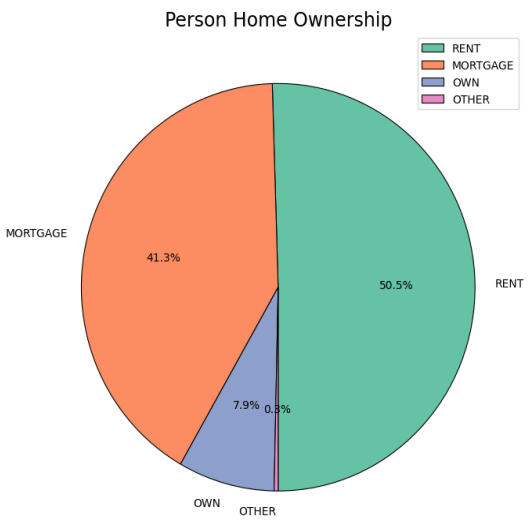


Figure 3: Person Home Ownership Pie Plot

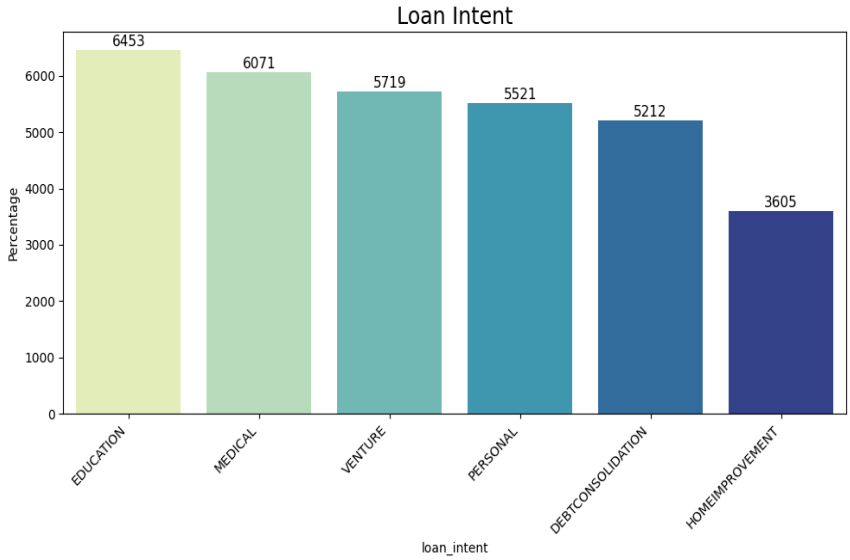


Figure 4: Loan Intent Bar Chart

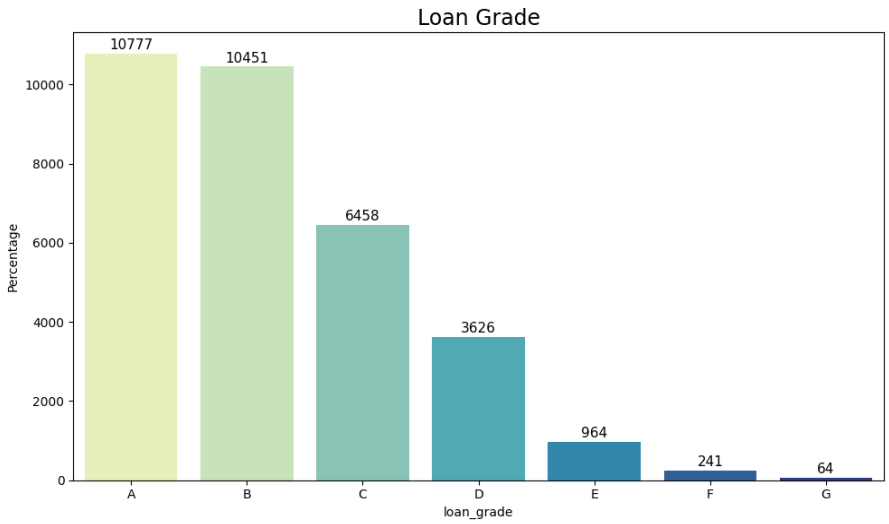


Figure 5: Loan Grade Bar Chart

Some of the Key EDA Insights are:

1) Most of the people taking a loan doesn't own their own house. They live on rent or have a mortgage.
2) Loan Intents such as Education, Medical, Venture, and Personal are almost equally distributed.
3) Loan Grade A and B comprises of more than half of the samples, indicating majority of the borrowers are considered under lower-risk category by the lenders.

Along with that, we have also observed some insights from the histograms and boxplots of numerical features:

1) Most borrowers are young adults in their 20s and 30s, with a few unrealistic outliers above the age of 90(such as 94, 123, 144)
2) Employment length is mostly between 0–10 years, with a few rare outliers exceeding 60 years.
3) Interest rates mostly fall between 7% and 14%, with a few borrowers receiving unusually high rates.
4) Credit history length is generally between 2–10 years, with some long-term outliers up to 30 years.

Overall, the dataset seems good, except few outliers in 'person_age' and 'person_emp_length' column which we will handle further along with null values and duplicates. Lastly, we observed the target variable which shows a clear class imbalance: majority of the samples are non-defaults(0).

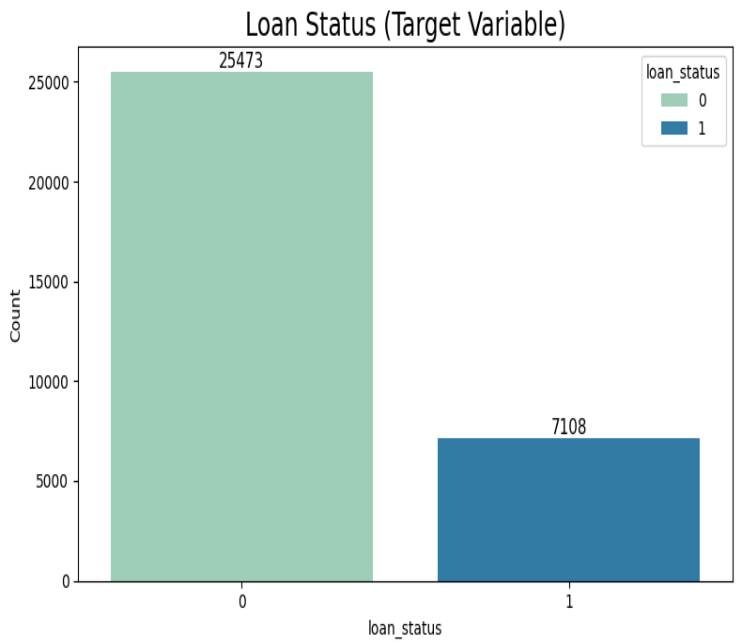


Figure 6: Target Variable (loan_status)

3. Data Cleaning and Preprocessing:

In this stage, we handled all the missing values, duplicates, and outliers. First, we removed outliers from the "person_age" and "person_emp_length" columns by filtering out unrealistic ages (greater than 90 years) and unrealistic employment lengths (greater than 60 years). After that, we replaced all missing values (895 in "person_emp_length" and 3,114 in "loan_int_rate") with their respective median values, since the majority of observations were close to the median and it better represented the central tendency of the data. Finally, after dropping 165 duplicate records, we were left with a total of 32,408 cleaned samples for further analysis.

Data Preprocessing

```
[25]
credit_risk_df.shape
(32581, 12)

[26]
# Removing unnecessary samples from person_age column(Filter for Age Feature)
credit_risk_df = credit_risk_df.drop(credit_risk_df[credit_risk_df["person_age"]> 90].index, axis=0)

[27]
credit_risk_df.shape
(32575, 12)

[28]
# Removing unnecessary samples from person_emp column (Filter for Employment Feature)
credit_risk_df = credit_risk_df.drop(credit_risk_df[credit_risk_df["person_emp_length"]>60].index, axis=0)

[29]
credit_risk_df.shape
(32573, 12)

[30]
# Dealing with Missing Value:
credit_risk_df.isnull().sum()
```

	0
person_age	0
person_income	0
person_home_ownership	0
person_emp_length	895
loan_intent	0
loan_grade	0
loan_amnt	0
loan_int_rate	3114
loan_status	0
loan_percent_income	0
cb_person_default_on_file	0
cb_person_cred_hist_length	0

dtype: int64

Figure 7: Handling Outliers

```
[32]
# Replacing missing values for loan_int_rate with the median value:
credit_risk_df.fillna({"loan_int_rate":credit_risk_df["loan_int_rate"].median()},inplace=True)

[33]
# Replacing missing values for person_emp_length with the median value:
credit_risk_df.fillna({"person_emp_length":credit_risk_df["person_emp_length"].median()},inplace=True)

[34]
credit_risk_df.isnull().sum()
```

	0
person_age	0
person_income	0
person_home_ownership	0
person_emp_length	0
loan_intent	0
loan_grade	0
loan_amnt	0
loan_int_rate	0
loan_status	0
loan_percent_income	0
cb_person_default_on_file	0
cb_person_cred_hist_length	0

dtype: int64

```
[35]
credit_risk_df.shape
(32573, 12)

[36]
# Removing the duplicates:
credit_risk_df.duplicated().sum()
np.int64(165)

[37]
credit_risk_df.drop_duplicates(inplace=True)
credit_risk_df.shape
(32408, 12)
```

Figure 8: Handling Null and Duplicate values

4. Feature Engineering:

To improve the model performance, we added additional three new meaningful features. Those are basically ratio-based features which are created from the existing columns to improve the predictive power of the model. Here are those features:

1) loan_to_income_ratio = loan_amnt / person_income
2) loan_to_emp_length_ratio = person_emp_length / loan_amnt
3) int_rate_to_loan_amt_ratio = loan_int_rate / loan_amnt

These ratios help the model understand the relationship between loan burden, income, employment length, and interest rate.

Creating New Columns:

```
# loan-to-income ratio
credit_risk_df['loan_to_income_ratio'] = credit_risk_df['loan_amnt'] / credit_risk_df['person_income']

# loan-to-employment length ratio
credit_risk_df['loan_to_emp_length_ratio'] =  credit_risk_df['person_emp_length']/ credit_risk_df['loan_amnt']

# Create interest rate-to-loan amount ratio
credit_risk_df['int_rate_to_loan_amt_ratio'] = credit_risk_df['loan_int_rate'] / credit_risk_df['loan_amnt']
```

```
credit_risk_df.head()
```

ome	person_home_ownership	person_emp_length	loan_intent	loan_grade	loan_amnt	loan_int_rate	loan_status	loan_percent_income	cb_person_default_on_file	cb_person_cred_hist_length	loan_to_income_ratio	loan_to_emp_length_ratio	int_rate_to_loan_amt_ratio
600	OWN	5.0	EDUCATION	B	1000	11.14	0	0.10	N	2	0.104167	0.005000	0.011140
500	MORTGAGE	1.0	MEDICAL	C	5500	12.87	1	0.57	N	3	0.572917	0.000182	0.002340
500	RENT	4.0	MEDICAL	C	35000	15.23	1	0.53	N	2	0.534351	0.000114	0.000435
400	RENT	8.0	MEDICAL	C	35000	14.27	1	0.55	Y	4	0.643382	0.000229	0.000408
900	OWN	2.0	VENTURE	A	2500	7.14	1	0.25	N	2	0.252525	0.000800	0.002856

Next steps: Generate code with credit_risk_df New interactive sheet

Figure 9: Adding 3 new features

Finally, after performing all the preprocessing, and feature engineering steps, we got our cleaned dataset with 32,408 samples and 15 features. Now, we will further perform encoding on categorial features and feature scaling on numerical features.

5. Data Splitting, Encoding & Feature Scaling:

After feature engineering, we first performed a train-test split of 75-25 in which 75% is used for training and 25% for testing. This step was performed earlier to ensures that all the tasks like encoding and scaling, are learned only from training data and note influenced by the test data. This prevents data leakage and also ensures unbiased model training. For categorical variables, we applied One-Hot Encoding because it converts non-numeric columns to binary columns without even introducing ordinality. And for numerical variables, we used StandardScaler to standardize the features so that columns operate on a similar scale. After completing encoding and scaling, both the transformed training and test datasets were ready for model fitting.

OneHot-Encoding on Categorical Features

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder()
ohe.fit(X_train[onehot_enco_columns])
```

```
OneHotEncoder
OneHotEncoder()
```

```
X_train_ohe = ohe.transform(X_train[onehot_enco_columns]).toarray()
X_test_ohe  = ohe.transform(X_test[onehot_enco_columns]).toarray()

X_train_ohe_df = pd.DataFrame(X_train_ohe, columns=ohe_feature_names, index=X_train.index)
X_test_ohe_df  = pd.DataFrame(X_test_ohe,  columns=ohe_feature_names, index=X_test.index)

# Concatening the OneHot-Encoded Columns with main dataframe and droping the old categorical columns
X_new_train = pd.concat([X_train_ohe_df,X_train],axis=1)
X_new_train = X_new_train.drop(onehot_enco_columns,axis=1)

X_new_train.head()
```

	person_home_ownership_MORTGAGE	person_home_ownership_OTHER	person_home_ownership_OWN	person_home_ownership_RENT	loan_intent_DEBTCONSOLIDATION	loan_intent_EDUCATION	loa
0	0.0	0.0	1.0	0.0	0.0	0.0	
1	1.0	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	1.0	0.0	1.0	
3	1.0	0.0	0.0	0.0	0.0	0.0	
4	1.0	0.0	0.0	0.0	0.0	0.0	

5 rows × 29 columns

Figure 10: One-Hot Encoding for Categorical Features

Scaling on Numerical Features

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaler.fit(X_new_train[scaler_columns])  # Fitting on Training data
```

```
StandardScaler
StandardScaler()
```

```
X_new_train[scaler_columns] = scaler.transform(X_new_train[scaler_columns])
X_new_test[scaler_columns] = scaler.transform(X_new_test[scaler_columns])
```

```
X_new_train.head()
```

ı_intent_MEDICAL	loan_intent_PERSONAL	loan_intent_VENTURE	...	person_age	person_income	person_emp_length	loan_amnt	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	loan_to_income_ra
0.0	0.0	1.0	...	-0.923620	-0.735034	-0.944106	0.435448	-1.424051	2.528280	-0.939507	2.511
0.0	0.0	1.0	...	-0.440142	1.149741	-0.440657	0.070026	-1.628613	-0.843037	-0.693003	-0.844
0.0	0.0	0.0	...	-0.278982	-0.509639	-0.440657	-0.883247	-1.693553	-0.655742	-0.446498	-0.648
0.0	0.0	0.0	...	-0.117823	0.566821	1.069688	2.453211	-0.005111	0.842621	-0.199994	0.866
0.0	0.0	1.0	...	-0.440142	0.819419	0.817964	0.864421	-1.424051	-0.281151	-0.939507	-0.294

```
X_new_test.head()
```

ı_intent_MEDICAL	loan_intent_PERSONAL	loan_intent_VENTURE	...	person_age	person_income	person_emp_length	loan_amnt	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	loan_to_income_ra
0.0	0.0	0.0	...	-0.278982	0.197638	0.314515	-0.247732	0.803393	-0.562094	-0.446498	-0.608
0.0	0.0	0.0	...	1.493772	-0.836074	3.335206	-0.406610	-1.615625	1.310860	1.525535	1.275
0.0	0.0	0.0	...	-0.923620	-0.608735	-0.692381	-0.724368	1.910622	-0.281151	-0.939507	-0.237
0.0	0.0	0.0	...	1.654931	0.663896	4.342103	2.453211	1.283950	0.748974	1.772039	0.743
1.0	0.0	0.0	...	1.010293	-0.035530	-0.692381	-0.247732	-1.329888	-0.374799	-0.199994	-0.423

Figure 11: StandardScaling for Numerical Features

6. Model Selection, Training & Evaluation:

After the stages of data preprocessing, feature engineering, encoding and scaling, we finally got the dataset where we will apply different machine learning classifiers on the credit risk data for training and model evaluation. In this stage, we used four classifiers: Logistic Regression, Random Forest, XGBoost and CatBoost. Here are few details regarding those models like what are those and how they work:

A) Logistic Regression Classifier:

It is a simple yet effective linear model for binary classification problems. This model basically calculates the probability that a data point belongs to a particular class. It uses logistic (sigmoid) function to model the relationship between input features and class probabilities, thus gives the output value between 0 and 1. If the predicted probability exceeds the threshold(usually 0.5) then the model classifies as positive class. It is easy to interpret, understand, and mainly works well when the relation is mostly linear

B) Random Forest Classifier:

Random Forest is an ensemble learning model that combines the result of multiple decision trees to get the final output. It uses random sampling and feature selection which simply means each tree is trained on random subset of data and a ransom subset of features, thus increasing the diversity among trees, reduces the overfitting and improving the model generalization. In this, final prediction is based on majority voting. Random Forest can capture non-linear relationships, and are robust to noise and missing values, thus making it highly accurate and stable.

C) XGBoost Classifier:

XGBoost short for Extreme Gradient Boosting is an advanced machine learning algorithm designed for efficiency, speed and higher performance. It is an ensemble learning method that combines multiple weak models to form a stronger model. XGBoost uses decision trees as its base learners and combines them sequentially to improve the model's performance. Each new tree is trained to correct the errors made by the previous tree and this process is called boosting. It uses gradient descent to minimize the loss function, and also uses regularization to prevent overfitting. XGBoost is one of the widely used models in different domain datasets as it handles complex non-linear patterns, support missing values, and performs pretty well on structured datasets.

D) CatBoost Classifier:

CatBoost known as Categorical Boosting, is another gradient bosting algorithm designed to handle categorical variables. It is based on the concept of gradient boosting technique where decision trees are built sequentially to minimize errors and improve predictions. The process works by constructing a decision tree and evaluating how much error are there in predictions. Once the first tree is built the next tree is created to correct the errors made by the previous one. This process continues iteratively with each new tree focusing on improving the model's predictions by reducing previous errors this process continues till a predefined number of iterations met. The result is an ensemble of decision trees that work together to provide accurate predictions. Its fast-training speed, built-in handling of feature importance, and strong performance make it a reliable choice for financial and credit-risk prediction tasks.

Finally, we trained four machine learning classifiers: Logistic Regression, Random Forest, XGBoost, and CatBoost, on our cleaned and preprocessed dataset. Each model was trained on the training data and further evaluated on the unseen test data to avoid any kind of biased performance of the models. Firstly, we started training the Logistic Regression Classifier Model with default parameters, which served as our baseline model. After that, we trained the Random Forest Classifier that provided a strong benchmark using class weighting to address the issue of class imbalance. We then implemented XGBoost and CatBoost which are both gradient-boosted tree algorithms. They are known for handling tabular data effectively and also for capturing complex non-linear relationships.

For model evaluation, we used standard classification performance metrics including Accuracy, Precision, Recall, F1-Score and Specificity. In this business use case, we specially put more emphasis on Recall and F1-Score because here our main priority was to accurately predict the minority(default) class. These metrics were computed for each model to understand their ability to detect loan defaulters, which is the key objective of credit-risk prediction.

1. Logistic Regression Classifier

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,precision_score,r

logit = LogisticRegression()
logit.fit(X_new_train,y_train)

LogisticRegression
LogisticRegression()

logit.score(X_new_train,y_train)

0.8744342960585864
```

Figure 12: Logistic Regression Classifier

```
logit_pred = logit.predict(X_new_test)

logit_pred

array([0, 0, 1, ..., 0, 1, 0])

print(classification_report(y_test,logit_pred))

              precision    recall  f1-score   support

           0       0.89      0.95      0.92      6330
           1       0.78      0.57      0.66      1772

    accuracy                           0.87      8102
   macro avg       0.83      0.76      0.79      8102
weighted avg       0.86      0.87      0.86      8102
```

Figure 13: Classification Report

2. Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=300,max_depth=None, random_state=42, class_weight="balanced")

rf.fit(X_new_train,y_train)

RandomForestClassifier
RandomForestClassifier(class_weight='balanced', n_estimators=300,
                       random_state=42)

rf.score(X_new_train,y_train)

1.0
```

Figure 14: Random Forest Classifier

```
rf.score(X_new_train,y_train)

1.0

rf_pred = rf.predict(X_new_test)
rf_pred

array([0, 0, 1, ..., 0, 1, 0])

print(classification_report(y_test,rf_pred))

              precision    recall  f1-score   support

           0       0.93      0.99      0.96      6330
           1       0.98      0.72      0.83      1772

    accuracy                           0.93      8102
   macro avg       0.95      0.86      0.89      8102
weighted avg       0.94      0.93      0.93      8102
```

Figure 15: Classification Report

3. XGBoost Classifier

```
from xgboost import XGBClassifier

# For handling imabalance data:
class_counts = y_train.value_counts()
neg, pos = class_counts[0], class_counts[1]
scale_pos_weight = neg / pos

xgb = XGBClassifier(scale_pos_weight=scale_pos_weight)

xgb.fit(X_new_train,y_train)

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=None,
              n_jobs=None, num_parallel_tree=None, ...)

xgb.score(X_new_train,y_train)

0.9743684686908582
```

Figure 16: XGBoost Classifier

```
xgb_pred = xgb.predict(X_new_test)
xgb_pred

array([0, 0, 1, ..., 0, 1, 0])

print(classification_report(y_test,xgb_pred))

              precision    recall  f1-score   support

           0       0.95      0.96      0.95      6330
           1       0.84      0.80      0.82      1772

    accuracy                           0.92      8102
   macro avg       0.89      0.88      0.89      8102
weighted avg       0.92      0.92      0.92      8102
```

Figure 17: Classification Report

```
from catboost import CatBoostClassifier
```

```
# Class Weights (for handling imabalance data):
class_counts = y_train.value_counts()
neg, pos = class_counts[0], class_counts[1]
class_weights = [1, neg/pos]
```

```
# Model training
cat = CatBoostClassifier(class_weights=class_weights,verbose=0)
cat.fit(X_new_train,y_train)
```

```
<catboost.core.CatBoostClassifier at 0x7903df86c890>
```

```
print(cat.score(X_new_train,y_train))
```

```
0.9551139636303794
```

Figure 18: CatBoost Classifier

```
cat_pred = cat.predict(X_new_test)
cat_pred
```

```
array([0, 0, 1, ..., 1, 1, 0])
```

```
print(classification_report(y_test,cat_pred))
```

```
              precision    recall  f1-score   support

           0       0.94      0.96      0.95      6330
           1       0.85      0.80      0.82      1772

    accuracy                           0.92      8102
   macro avg       0.90      0.88      0.89      8102
weighted avg       0.92      0.92      0.92      8102
```

Figure 19: Classification Report

RESULTS & COMPARISON

A. Logistic Regression Classifier:

Logistic Regression was used as the baseline linear classifier. It achieved an accuracy of 87%, precision score of 77.9%, recall score of 56.7%, and f1-score of 65.6%. Here, the model accuracy was good, but our main focus was to put more emphasis on correctly classifying the defaulters for which we were counting more on recall of the model. In Logistic Regression Classifier, the recall for the minority class(defaulters) was relatedly low(56.7%), indicating that many defaulters were misclassified.

```
[69]
print(classification_report(y_test,logit_pred))
```

```
              precision    recall  f1-score   support

           0       0.89      0.95      0.92      6330
           1       0.78      0.57      0.66      1772

    accuracy                           0.87      8102
   macro avg       0.83      0.76      0.79      8102
weighted avg       0.86      0.87      0.86      8102
```

```
[70]
accuracy_score(y_test,logit_pred)
```

```
0.870278943470748
```

```
[71]
precision_score(y_test,logit_pred)
```

```
0.7792408985282726
```

```
[72]
recall_score(y_test,logit_pred)
```

```
0.5677200902934537
```

Figure 20: Logistic Regression Classifier Performance

B. Random Forest Classifier:

This model was far better than the Logistic Regression Classifier, as it not only got an accuracy of 93.4% but also had a precision score of 97.5%, recall score of 71.6% and f1score of 82.6%. This is because Random Forest Classifier, being an ensemble bagging model, was able to capture more complex relations and improved the overall performance and the predictive strength. Here recall is 0.71 which sill suggests that the mdel fails to detect a noticeable portion of defaulters.

```
print(classification_report(y_test,rf_pred))

              precision    recall  f1-score   support

           0       0.93      0.99      0.96      6330
           1       0.98      0.72      0.83      1772

    accuracy                           0.93      8102
   macro avg       0.95      0.86      0.89      8102
weighted avg       0.94      0.93      0.93      8102

accuracy_score(y_test,rf_pred)

0.9340903480622068

precision_score(y_test,rf_pred)

0.9754224270353302

recall_score(y_test,rf_pred)

0.7167042889390519
```

Figure 21: RandomForest Classifier Performance

C. XGBoost Classifier:

XGBoost demonstrated excellent performance by combining boosting with regularization. With 80% recall, 82% F1-score, and 92.3% accuracy, it delivered the strongest minority-class detection while keeping overall performance high. Its ability to handle non-linear feature interactions and class imbalance makes it the best-performing model in this project.

```
[89] print(classification_report(y_test,xgb_pred))

              precision    recall  f1-score   support

           0       0.95      0.96      0.95      6330
           1       0.84      0.80      0.82      1772

    accuracy                           0.92      8102
   macro avg       0.89      0.88      0.89      8102
weighted avg       0.92      0.92      0.92      8102

[90] accuracy_score(y_test,xgb_pred)

0.9231054060725746

[91] precision_score(y_test,xgb_pred)

0.8397397989355411

[92] recall_score(y_test,xgb_pred)

0.801354401805869
```

Figure 22: XGBoost Classifier Performance

D. CatBoost Classifier:

CatBoost also performed strongly with an accuracy of 92% and balanced precision/recall values. Although CatBoost was close to XGBoost in performance, its recall (0.79) was slightly lower, making it the second-best model for default classification in this study.

```
[00] print(classification_report(y_test,cat_pred))

              precision    recall  f1-score   support

           0       0.94      0.96      0.95      6330
           1       0.85      0.80      0.82      1772

    accuracy                           0.92      8102
   macro avg       0.90      0.88      0.89      8102
weighted avg       0.92      0.92      0.92      8102

[01] accuracy_score(y_test,cat_pred)

0.9239693902740064

[02] precision_score(y_test,cat_pred)

0.8465227817745803

[03] recall_score(y_test,cat_pred)

0.7968397291196389
```

Figure 23: CatBoost Classifier Performance

COMPARISON:

Across all four models, we can clearly see the performance differences. Logistic Regression served as a simple baseline and it struggled in identifying the minority class(defaulters) due to its linear nature, thus resulting in the lowest recall and F1-Score. Next, Random Forest improved the results a lot compared to the baseline model. Regardless of the high accuracy and precision, Random Forest model still had a moderate recall (71%), indicating the limited effectiveness in capturing the minority class. After that, we trained XGBoost and CatBoost, both are boosting models and successfully outperformed the others by effectively handling the non-linear patterns and class-imbalance. Out of those two, XGBoost slightly outperformed the CatBoost because it achieved the overall best balance. XGBoost has the highest recall for default cases while maintaining strong accuracy and F1-score, making it the most reliable model for credit-risk prediction.

	Model	Accuracy	Precision	Recall	F1 Score
1	Logistic Regression	0.870279	0.779241	0.567720	0.656872
2	Random Forest	0.934090	0.975422	0.716704	0.826285
3	XGBoost	0.923105	0.839740	0.801354	0.820098
4	CatBoost	0.923969	0.846523	0.796840	0.820930

Figure 24: Model Comparison Table

BEST MODEL (XGBoost Classifier):

XGBoost comes up to be the best model for this credit risk prediction dataset because it achieved the highest recall of 80% for the minority class while maintaining strong precision and overall accuracy. The model used scale_pos_weight to compensate for the issue of class imbalance. Also, the model's boosting approach made it capture complex non-linear relations. Further, when we checked the feature importance analysis, it was evident that XGBoost effectively utilized the key loan-risk indicators such as income, interest rate, debt-to-income ratio, and credit history length. Additionally, the ROC-AUC score of **0.95** indicates excellent discriminative power. This combination of high recall, balanced F1-score, and strong interpretability makes XGBoost the most suitable model for credit-risk prediction.

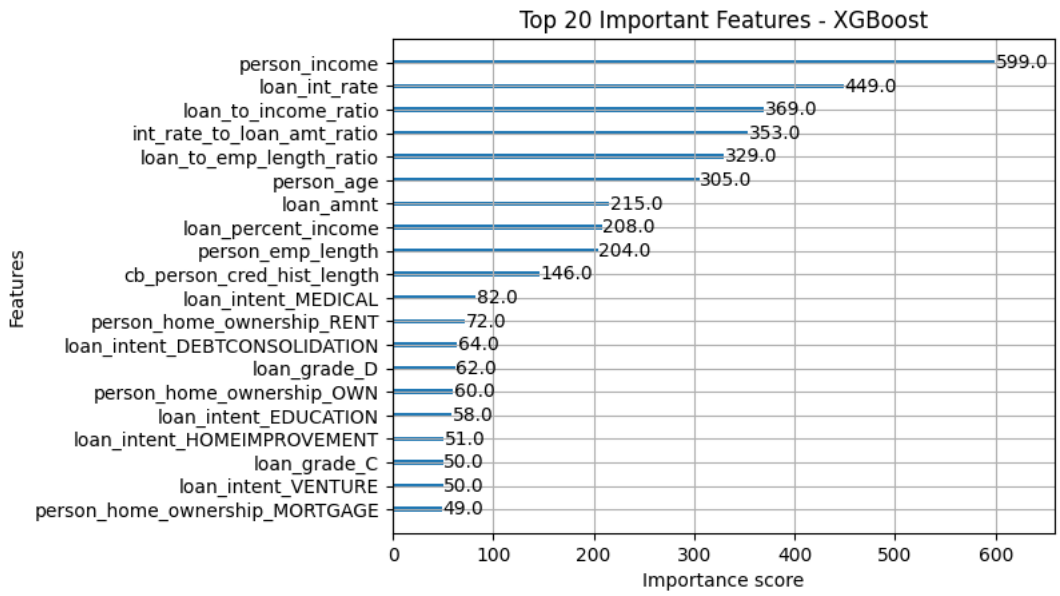


Figure 25: Feature Importance (XGBoost Classifier)

Key Insight:

Based on the above feature importance plot of XGBoost, **person_income, loan_int_rate, loan_to_income_ratio, int_rate_to_loan_amt_ratio, loan_to_emp_length_ratio, person_age** and **loan_amnt** are the core features that strongly influence whether a borrower defaults on loan or not.

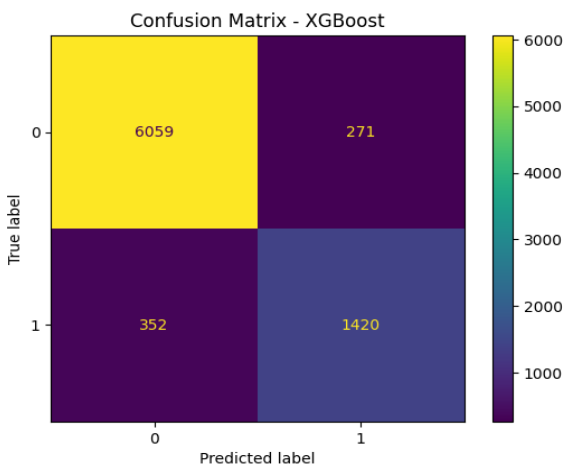
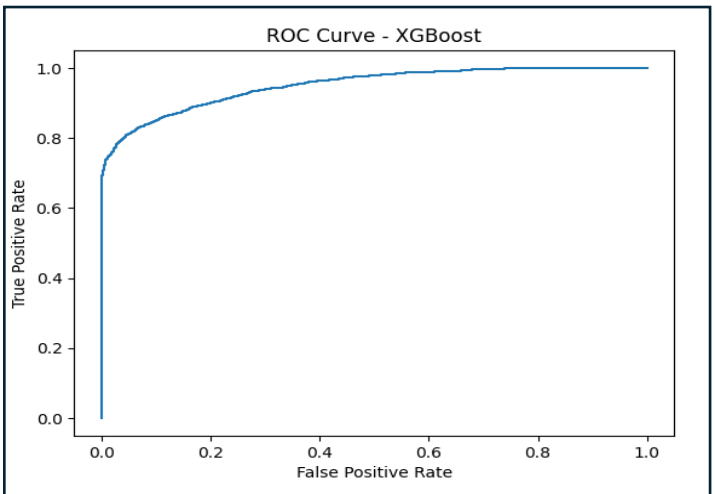


Figure 26: Confusion Matrix



AUC Score: 0.9494419065

Figure 27: ROC Curve

ROC Curve Interpretation:

- The AUC-ROC curve shows how well the model separates:
  Class 0 → Non-default
  Class 1 → Default
- AUC = 0.92 means:
  The model is 92% likely to rank a random defaulter higher risk than a random non-defaulter. The model has excellent discriminatory power, and it is perfectly suitable for real-world credit screening.

BUSINESS INSIGHTS

Here are some actionable insights from the model:

1) Borrowers with low income or high loan to income ratio are at significantly higher risk of default.
2) High Interest Rates are linked to higher default. It can be concluded from the loan_int_rate predictor because that comes under one of the top predictors in feature importance list.
3) Credit history length is also important because longer and cleaner the credit history, lesser will be the chance of default.
4) Banks can use this XGBoost model to screen the risky applicants, adjust interest rates, improve the approval decisions and even reduce the financial losses.

CONCLUSION

In this project, we developed a complete credit-risk prediction system using multiple machine learning algorithms to classify borrowers as defaulters or non-defaulters. After performing data cleaning, feature engineering, and preprocessing, we finally trained and evaluated four classification models: Logistic Regression, Random Forest, XGBoost, and CatBoost. For model evaluation, we used several performance metric parameters, including accuracy, precision, recall, F1-score and specificity. In our model evaluation process, we prioritize correctly classifying the defaulters(minority class). Out of all models, XGBoost came out to be the best classifier that achieves the highest recall of 80% and F1-Score of 82% for the minority class, which is very crucial for minimizing false negatives in credit risk management.

After selecting out our best classifier, we performed feature importance analysis to filter out the top features from the model. This analysis concluded that income, interest rate, loan-to-income ratio, and credit history length were the most essential predictors. Further, model's ROC curve (AUC=0.95) proved the strong discriminative ability. Moreover, the model can be further improved by performing hyperparameter tuning and by experimenting with cost-sensitive learning to penalize misclassification of high-risk borrowers.